MODUL PRAKTIKUM | PEMROGRAMAN INTERNET

Mengamankan REST API Pemantauan Sensor dengan JWT

Python | FastAPI | SQLite | Swagger UI

CPMK-5: Pembuatan API dan Keamanan

Dari data pengukuran ke layanan yang aman

CLIENT	REST API	JWT + ROLE	DATABASE
Swagger / Postman	Sensor dan pengukuran	Identitas dan izin	SQLite

Modul ini melanjutkan praktikum CPMK-4. Mahasiswa mengamankan API sensor yang sudah mengenal operasi CRUD, lalu menguji bahwa hak akses berbeda untuk admin, operator, dan viewer.

Konteks Teknik Elektro
Data latihan berupa pembacaan tegangan pada Panel A dan arus pada Panel B. Nilai dibuat simulasi dan tidak terhubung ke perangkat listrik nyata.

Program Studi Teknik Elektro

1. Tujuan dan capaian praktikum

CPMK-5 menargetkan kemampuan membangun dan mendokumentasikan RESTful API serta menerapkan autentikasi dan otorisasi dasar menggunakan JWT. Praktikum ini memakai Python sebagai pengganti Java/Spring Security.

Setelah praktikum, mahasiswa dapat

- Menjelaskan hubungan request HTTP, response JSON, dan operasi CRUD pada API.
- Meminta access token melalui endpoint login dan mengirimnya sebagai Bearer token.
- Membedakan autentikasi (siapa pengguna) dari otorisasi (aksi yang boleh dilakukan).
- Menerapkan hash password dan pemeriksaan role pada endpoint sensor.
- Menguji status 200, 201, 401, 403, 404, dan 422 melalui Swagger UI atau Postman.

Prasyarat

Mahasiswa sudah memahami JSON, metode GET/POST/PUT/DELETE, relasi Sensor-Measurement, dan CRUD dari praktikum CPMK-4. SQLModel dan SQLite tetap digunakan agar fokus belajar berpindah ke keamanan API.

Pertanyaan pemantik
Mengapa API pencatat pengukuran perlu membedakan operator yang menambah data dari viewer yang hanya membaca hasil?

2. Skenario dan aturan akses

Bayangkan laboratorium memiliki sensor tegangan dan sensor arus. Operator boleh memasukkan hasil pembacaan. Viewer hanya melihat data. Admin mengelola sensor dan dapat memperbaiki atau menghapus data yang keliru.

Peran	Membaca	Tambah pengukuran	Kelola sensor	Ubah/hapus pengukuran
admin	Ya	Ya	Ya	Ya
operator	Ya	Ya	Tidak	Tidak
viewer	Ya	Tidak	Tidak	Tidak

Data demo pertama

ID contoh	Kode	Parameter	Lokasi	Pengukuran awal
1	S-01	tegangan	Panel Praktikum A	220.5 V
2	S-02	arus	Panel Praktikum B	Belum ada

Penting
ID sensor dibuat otomatis oleh SQLite. Gunakan ID dari response GET, bukan menganggap ID selalu sama pada database yang sudah pernah dipakai.

3. Peta file proyek

Setiap file memiliki tugas tertentu. Endpoint tidak langsung mengatur SQL, dan file keamanan dipisahkan dari file route agar alur belajar mudah diikuti.

File	Tugas utama
main.py	Membuat aplikasi FastAPI, menyiapkan tabel/data demo, memasang router.
config.py	Membaca SECRET_KEY, masa berlaku JWT, dan URL SQLite dari .env.
database.py	Membuat engine SQLite dan menyediakan sesi per request.
models.py / schemas.py	Model tabel database / bentuk request dan response JSON.
crud.py	Fungsi tambah, baca, ubah, hapus data sensor dan pengukuran.
security.py	Hash password, membuat token, memeriksa token, dan membatasi role.
seed.py	Membuat akun demo dan pengukuran awal pada database lokal.
routers/auth.py	Login OAuth2 password flow dan endpoint /auth/me.
routers/sensors.py	Endpoint sensor dan riwayat pengukuran milik sensor.
routers/measurements.py	Endpoint CRUD pengukuran dengan pemeriksaan role.

Aliran request

Client -> router -> dependency keamanan -> CRUD -> sesi database -> response JSON. Urutan ini memisahkan tugas tiap modul.

4. Langkah 1 - siapkan Python

Pastikan Python 3.10 atau lebih baru tersedia. Buka PowerShell dan masuk ke folder proyek.

```
cd C:\xampp\htdocs\pemrogramaninternet\cpmk5_jwt
py --version
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

Apa yang dilakukan perintah ini?

- cd berpindah ke folder proyek.

- py -m venv .venv membuat lingkungan paket khusus proyek.

- pip memasang FastAPI, Uvicorn, SQLModel, PyJWT, pwdlib, dan dukungan form login.

Jika perintah py tidak dikenal
Periksa instalasi Python dan PATH Windows. Alternatifnya, ganti py dengan python jika Python sudah dapat dipanggil dari PowerShell.

Peran paket utama

FastAPI membuat endpoint dan dokumentasi OpenAPI. SQLModel menghubungkan model Python ke SQLite. PyJWT menandatangani dan memverifikasi JWT. pwdlib membuat hash password menggunakan Argon2. Uvicorn menjalankan server lokal.

5. Langkah 2 - atur secret dan jalankan

Secret adalah kunci server untuk menandatangani token. Jangan menulis secret sungguhan ke source code atau membagikannya lewat repository.

```
Copy-Item .env.example .env
py -c "import secrets; print(secrets.token_urlsafe(32))"
```

Salin hasil acak ke baris SECRET_KEY di file .env. Simpan .env di folder proyek. ACCESS_TOKEN_EXPIRE_MINUTES=30 membuat token demo berlaku 30 menit.

```
SECRET_KEY=salin_hasil_secret_acak_ke_sini
ACCESS_TOKEN_EXPIRE_MINUTES=30
DATABASE_URL=sqlite:///./lab_sensor_secure.db
```

Jalankan server

```
.\.venv\Scripts\python.exe -m uvicorn main:app --reload
```

Biarkan terminal terbuka. Buka http://127.0.0.1:8000/docs di browser. FastAPI menyediakan halaman dokumentasi interaktif dari schema endpoint.

Bukan melalui Apache
Walaupun folder proyek berada di htdocs, aplikasi dijalankan oleh Uvicorn. XAMPP tidak perlu menjalankan file Python ini melalui Apache.

6. Konsep: autentikasi, JWT, otorisasi

Tahap	Yang terjadi
1. Login	Client mengirim username dan password ke POST /auth/token.
2. Verifikasi	Server membandingkan password dengan hash Argon2 di database.
3. Token	Server menerbitkan JWT berisi identitas sub dan waktu exp.
4. Request	Client mengirim Authorization: Bearer pada endpoint privat.
5. Izin	Server memverifikasi token, memuat role user, lalu mengizinkan/menolak aksi.

Kode respons yang perlu diamati

Status	Makna pada praktikum
200 / 201	Request berhasil dibaca / data baru berhasil dibuat.
401 Unauthorized	Belum login, password salah, token tidak valid, atau kedaluwarsa.
403 Forbidden	Sudah login tetapi role tidak mendapat izin untuk aksi itu.
404 Not Found	ID sensor atau pengukuran tidak ditemukan.
422 Unprocessable Entity	Isi request tidak sesuai nama field atau tipe data.

JWT bukan enkripsi

Token ditandatangani untuk membuktikan keaslian dan integritas. Payload dapat dibaca; jangan simpan password, secret, atau data rahasia di dalamnya.

7. Langkah 3 - login lewat Swagger

Aplikasi menyiapkan akun demo dan hash password pada database lokal. Gunakan akun berikut untuk memahami tiga tingkat izin.

Username	Password praktikum	Role
admin	AdminLab!2026	admin
operator	OperatorLab!2026	operator
viewer	ViewerLab!2026	viewer

Cara termudah

- Pada halaman /docs, tekan tombol Authorize di bagian atas.

- Isi username dan password, tekan Authorize, lalu Close.

- Jalankan GET /auth/me. Response menunjukkan username dan role aktif.

- Jalankan GET /sensors/. Jika token terkirim, API mengembalikan data JSON sensor.

Melihat endpoint login

POST /auth/token menerima form username/password, bukan JSON. Response berisi access_token dan token_type. Setelah tombol Authorize digunakan, Swagger mengirim token otomatis pada request berikutnya.

```
{
  "access_token": "eyJ...token...",
  "token_type": "bearer"
}
```

Jaga password
Password hanya dipakai saat login. Request berikutnya mengirim token, bukan mengulang password.

8. Langkah 4 - buktikan aturan role

Uji endpoint dengan akun berbeda. Catat response status dan jelaskan apakah penolakan terjadi karena belum terautentikasi atau tidak memiliki izin.

Login sebagai	Coba request	Hasil yang diharapkan
Tanpa token	GET /sensors/	401 Unauthorized
viewer	GET /sensors/	200 OK
viewer	POST /measurements/	403 Forbidden
operator	POST /measurements/	201 Created
operator	POST /sensors/	403 Forbidden
admin	POST /sensors/	201 Created

Langkah uji viewer

- Gunakan Authorize untuk login sebagai viewer.
- Buka POST /measurements/ dan kirim body yang valid.
- API mengenali token viewer, lalu role checker mengembalikan 403.
- Login sebagai operator dan ulangi body yang sama. Hasilnya 201 dan ada ID pengukuran baru.

Setelah mengganti akun
Jika Swagger masih memakai token lama, buka Authorize, pilih Logout, lalu login dengan akun yang baru.

9. Langkah 5 - CRUD pengukuran

Sebagai operator, tambahkan hasil simulasi sensor. Nilai dan unit berikut contoh praktikum, bukan hasil pembacaan alat nyata.

```
{
  "sensor_id": 1,
  "value": 221.4,
  "unit": "V",
  "measured_at": "2026-10-21T10:00:00"
}
```

Operasi	Endpoint	Peran
Create	POST /measurements/	admin atau operator
Read semua	GET /measurements/	semua yang login
Read satu	GET /measurements/{id}	semua yang login
Update	PUT /measurements/{id}	admin
Delete	DELETE /measurements/{id}	admin
Riwayat sensor	GET /sensors/{id}/measurements	semua yang login

Urutan CRUD yang disarankan

- Create: POST pengukuran dengan sensor_id dari GET /sensors/.

- Read: GET /measurements/ atau GET /sensors/1/measurements.

- Update: login admin; kirim seluruh field baru ke PUT /measurements/{id}.

- Delete: login admin; pilih ID yang benar lalu DELETE /measurements/{id}.

Relasi data
Pengukuran terhubung ke sensor melalui sensor_id. Jika ID sensor tidak ada, POST pengukuran mengembalikan 404.

10. Pengujian opsional dengan Postman

Materi RESTful sebelumnya mengenalkan JSON dan pengujian endpoint dengan Postman atau cURL. Swagger cukup untuk praktikum ini; Postman collection disertakan untuk mencoba request di luar /docs.

- Import praktikum_cpmk5.postman_collection.json.

- Import praktikum_cpmk5.postman_environment.json dan pilih environment lokal.

- Jalankan 1 - Login operator. Script response menyimpan access_token ke variabel token.

- Lanjutkan 2 - Profil pengguna, 3 - Daftar sensor, 4 - Tambah pengukuran, lalu 5 - Daftar pengukuran.

Bentuk header yang dikirim

```
Authorization: Bearer <access_token>
Accept: application/json
Content-Type: application/json
```

Contoh cURL untuk membaca data

```
curl -X GET "http://127.0.0.1:8000/sensors/1/measurements" \
  -H "accept: application/json" \
  -H "Authorization: Bearer TOKEN_HASIL_LOGIN"
```

Token adalah kredensial

Jangan tempelkan access_token ke laporan publik, screenshot yang dibagikan, atau repository. Token dapat dipakai sampai kedaluwarsa.

11. Membaca kode keamanan

Hash password

security.py memisahkan password hash dan verifikasi. Fungsi hash dipakai seed.py saat akun demo dibuat; endpoint login memanggil fungsi verifikasi.

```
password_hasher = PasswordHash.recommended()

def hash_password(password: str) -> str:
    return password_hasher.hash(password)

def verify_password(password: str, stored_hash: str) -> bool:
    return password_hasher.verify(password, stored_hash)
```

Token dan dependency

```
payload = {"sub": username, "exp": expires_at}
token = jwt.encode(payload, SECRET_KEY, algorithm="HS256")

current_user = Depends(get_current_user)
operator = Depends(require_roles("admin", "operator"))
```

Dependency get_current_user mengambil Bearer token, memverifikasi tanda tangan dan waktu kedaluwarsa, kemudian membaca pengguna aktif dari database. require_roles() membandingkan role terbaru yang tersimpan di database.

Mengapa role diambil dari database?

Agar perubahan role pada database berlaku pada request berikutnya tanpa menunggu token lama kedaluwarsa. Token demo cukup membawa subject dan expiration.

Batas rancangan
Praktikum belum mencakup refresh/revokasi token, pembatasan percobaan login, HTTPS, atau sistem manajemen secret untuk deployment.

12. Tugas dan penilaian

Tugas praktikum

- Buat catatan uji akses tiga role untuk endpoint baca dan tulis. Lampirkan minimal empat response status.
- Tambahkan satu sensor parameter daya dan dua pengukuran simulasi dalam satuan W.
- Ubah masa token menjadi 2 menit. Catat response setelah token kedaluwarsa.
- Jelaskan letak hash, pembuatan JWT, verifikasi token, dan pemeriksaan role dengan menunjuk file/fungsi.
- Tuliskan dua perbaikan keamanan sebelum API digunakan di jaringan nyata.

Rubrik ringkas

Aspek	Bobot	Bukti
API dan CRUD berfungsi	30%	Request JSON dan response sesuai endpoint.
Autentikasi JWT	25%	Login menghasilkan token; endpoint privat menolak tanpa token.
Otorisasi role	25%	Viewer/operator/admin mendapat izin yang berbeda.
Dokumentasi dan analisis	20%	Langkah reproduksi, status HTTP, dan refleksi keamanan.

Bukti minimum
Sertakan response GET berhasil, POST operator berhasil, POST viewer ditolak, dan penjelasan 401 vs 403.

13. Pemeriksaan akhir dan sumber

Checklist sebelum dikumpulkan

- Uvicorn aktif dan dokumentasi /docs dapat dibuka.
- File .env memiliki secret acak dan tidak dilampirkan ke laporan/repository.
- GET dengan token mengembalikan JSON dan status 200.
- POST pengukuran berhasil untuk operator dan ditolak untuk viewer dengan 403.
- Password tidak muncul pada response pengguna dan tersimpan sebagai hash.
- Nama file modular dan role yang diuji dijelaskan dalam laporan.

Troubleshooting cepat

Gejala	Pemeriksaan
Secret belum diatur	Salin .env.example ke .env dan ganti SECRET_KEY.
401	Login ulang; pastikan token masih berlaku dan ikut terkirim.
403	Token valid, tetapi role tidak punya izin endpoint.
422	Cek nama field, tipe angka, waktu ISO, dan body JSON.
Pengukuran kosong	Cek sensor_id atau lihat GET /measurements/.
Database berbeda	Jalankan server dari folder proyek yang berisi .env.

Sumber belajar

Materi kuliah: Pengenalan RESTful API dan Format JSON (PDF yang disediakan dosen). Dokumentasi resmi: FastAPI OAuth2, hashing password, dan JWT; SQLModel session sebagai dependency FastAPI.

Modul memakai data simulasi laboratorium. Nilai tegangan/arus bukan panduan bekerja pada rangkaian bertegangan. Akun demo dan secret harus diganti sebelum deployment.